

Documento técnico de código fuente

Proyecto: Cooperativa Financiera El Progreso

Norma: 220501096 — Desarrollar la solución de software de acuerdo con especificaciones de diseño y marcos de referencia

1. Organización del proyecto

ElProgreso/	(solución .slnx)
— ElProgreso.slnx	Archivo de solución
— README.md	Documentación principal del repositorio
— LICENSE	MIT
— docs/	Diagrama de clases (Draw.io + PDF)
— SENA/	Este anexo de evidencias
— ElProgreso/	Proyecto de consola (.csproj)
— Program.cs	Composition root / punto de entrada
— Models/	Entidades + reglas de negocio intrínsecas
— Member.cs	
— Movement.cs	
— MovementType.cs	
— Repositories/	Persistencia, detrás de una interfaz
— IMemberRepository.cs	
— InMemoryMemberRepository.cs	
— Services/	Casos de uso
— MemberService.cs	
— MovementService.cs	
— ReportService.cs	
— ExchangeRateService.cs	
— UI/	Consola: menús + entrada/salida
— TellerConsole.cs	
— ManagementReportsMenu.cs	
— ConsoleReader.cs	
— ConsoleFormat.cs	

Un único proyecto de consola, con las capas separadas por carpeta y por convención de dependencias (cada **using** respeta el sentido de las flechas del diagrama de clases), en vez de por múltiples **.csproj**. Con un solo ejecutable

consumidor, dividir en más proyectos habría sido complejidad sin beneficio real para el alcance del ejercicio.

2. Explicación de las capas / módulos

Models — el núcleo del dominio

No depende de ninguna otra capa del proyecto (solo de **System**). Aquí viven las reglas de negocio que nunca deben poder saltarse, sin importar quién llame al código:

```
// Models/Member.cs

public decimal Balance => _movements.Sum(m => m.SignedEffect);
```

Balance es una propiedad calculada, no un campo. No existe ningún **set** público para el saldo en todo el proyecto — es físicamente imposible asignarle un valor directamente, lo que cumple la regla "el saldo... no es un dato que se digite ni se corrija a mano" al nivel del propio modelo de datos, no como una validación que alguien podría olvidar en la UI.

```
// Models/Member.cs

public Movement RegisterWithdrawal(decimal amount)
{
    if (amount <= 0)
        throw new InvalidOperationException("El valor del retiro
debe ser mayor a cero.");

    var fee = amount > LargeWithdrawalThreshold ? CashHandlingFee
: 0m;

    var totalDebit = amount + fee;
```

```
        if (totalDebit > Balance)

            throw new InvalidOperationException(/* mensaje para la
cajera */);

        var movement = new Movement(MovementType.Withdrawal, amount,
fee, DateTime.Now);

        _movements.Add(movement);

        return movement;
    }
}
```

La comisión de manejo de efectivo y el rechazo por saldo insuficiente están dentro del método que registra el retiro, no en la capa de UI ni en el servicio. Cualquier punto de entrada futuro al sistema (una API REST, por ejemplo) heredaría automáticamente esta regla sin tener que reimplementarla.

Repositories — persistencia detrás de un contrato

```
// Repositories/IMemberRepository.cs

public interface IMemberRepository
{
    void Add(Member member);

    Member? GetByDocumentNumber(string documentNumber);

    List<Member> GetAll();

    List<Member> SearchByName(string namePart);

    bool ExistsByDocumentNumber(string documentNumber);
}
```

```
void Remove(Member member);  
}
```

InMemoryMemberRepository es la única implementación hoy (una **List<Member>** en memoria), pero todo el resto del sistema depende de la interfaz, nunca de esa clase concreta. Es el patrón Repository combinado con inversión de dependencias: cambiar a una base de datos real (ver **SENA/220501095.../04 Modelo de base de datos/Modelo de base de datos/Modelo de base de datos.png** o **SENA/220501095.../04 Modelo de base de datos/Modelo de base de datos/Modelo de base de datos.pdf**) es agregar una clase nueva y cambiar una línea en **Program.cs**.

Services — los casos de uso

Cada servicio tiene una responsabilidad y depende solo de lo que necesita:

```
// Services/MovementService.cs  
  
public sealed class MovementService(IMemberRepository  
memberRepository)  
{  
    public Movement RegisterDeposit(string documentNumber, decimal  
amount) =>  
        GetMemberOrThrow(documentNumber).RegisterDeposit(amount);  
    // ...  
}
```

Nótese el *primary* constructor (**MovementService(IMemberRepository memberRepository)**), una característica de C# 12+ que evita escribir a mano el campo privado y el constructor tradicional para casos simples de inyección de dependencias.

ExchangeRateService es el único servicio que habla con el exterior (**HttpClient**), y absorbe cualquier falla para que nunca escale hacia arriba:

```
// Services/ExchangeRateService.cs

public async Task<ExchangeRate?> GetLastestRateAsync()
{
    try
    {
        var records = await
httpClient.GetFromJsonAsync<List<TrmRecord>>(RequestUri);

        // ... valida y parsea

        return new ExchangeRate(rate, record.ValidFrom.Value,
record.ValidUntil.Value);
    }

    catch (Exception ex) when (ex is HttpRequestException or
TaskCanceledException or JsonException)

    {
        return null;    // nunca deja caer la aplicación por una
falla de red
    }
}
```

ReportService concentra los 6 informes de gerencia, cada uno como una consulta LINQ declarativa sobre los asociados del repositorio (ver fragmento de "¿Quiénes son mis mejores asociados?"):

```
// Services/ReportService.cs

public List<TopMember> GetTopMembers(int count = 5) =>

    memberRepository.GetAll()

        .OrderByDescending(m => m.Balance)

        .Take(count)

        .Select(m => new TopMember(m.DocumentNumber, m.FullName,
m.Balance))

        .ToList();
```

UI — la única capa que toca Console

```
// UI/TellerConsole.cs

private void RegisterMember()
{
    var documentNumber = ConsoleReader.ReadRequiredText("Número de
documento: ");

    var fullName = ConsoleReader.ReadRequiredText("Nombre
completo: ");

    // ...

    try
    {
        var member = memberService.Register(documentNumber,
fullName, phoneNumber, address);

        Console.WriteLine($"Asociado registrado. {member.FullName}
queda con la cuenta en {ConsoleFormat.Cop0(0)}.");
    }
}
```

```

    }

    catch (Exception ex) when (ex is ArgumentException or
InvalidOperationException)

    {

        Console.WriteLine($"No se pudo registrar: {ex.Message}");

    }

}

```

Cada opción del menú sigue el mismo patrón: leer entrada validada (**ConsoleReader**), llamar al **Service** correspondiente, y capturar las excepciones de dominio para mostrarlas como texto simple — nunca un stack trace llega a la cajera.

Program.cs — composition root

```

var httpClient = new HttpClient { BaseAddress = new
Uri("https://www.datos.gov.co/"), Timeout =
TimeSpan.FromSeconds(8) };

var memberRepository = new InMemoryMemberRepository();
var exchangeRateService = new ExchangeRateService(httpClient);
var memberService = new MemberService(memberRepository,
exchangeRateService);
var movementService = new MovementService(memberRepository);
var reportService = new ReportService(memberRepository);

var console = new TellerConsole(memberService, movementService,
reportService);

```

```
await console.RunAsync();
```

Es el único lugar del proyecto donde se usan constructores de clases concretas (**new InMemoryMemberRepository()**, **new HttpClient()**); todo lo demás recibe sus dependencias por constructor (inyección manual, sin contenedor DI — no se justifica uno para una app de consola de este tamaño).

3. Tecnologías y frameworks utilizados

Tecnología	Uso en el proyecto
.NET 10 / C# 14	Runtime y lenguaje
System.Net.Http.HttpClient	Consumo asíncrono de la API de la TRM
System.Text.Json (GetFromJsonAsync<T>)	Deserialización de la respuesta de la TRM
System.Text.RegularExpressions ([GeneratedRegex])	Validación de documento/nombre/teléfono, compilada en build time
LINQ	Búsquedas, ordenamientos y agregaciones (informes, saldo calculado)
System.Globalization (CultureInfo)	Formato de moneda es-CO (pesos) y en-US (dólares) y parseo tolerante de decimales

No se usan librerías de terceros ni NuGet packages externos — todo lo necesario ya viene en el framework.

4. Buenas prácticas de programación aplicadas

- **Nomenclatura en inglés en todo el código** (clases, métodos, variables, carpetas) — solo los textos que ve la cajera están en español, siguiendo la convención pedida en el enunciado.
- **Inmutabilidad donde corresponde:** **Movement** es un **record** (igualdad por valor, sin setters); en **Member**, **DocumentNumber** y **JoinedAt** son de solo lectura tras la construcción.
- **Encapsulación estricta:** **Member._movements** es **private readonly**; se expone como **ReadOnlyList<Movement>** para que nadie fuera de **Member** pueda añadir o quitar movimientos directamente.
- **sealed en clases sin escenario real de herencia** (**Member**, **Movement**, los **Service**, **InMemoryMemberRepository**), declarando explícitamente que no son puntos de extensión.
- **Manejo de errores por excepciones tipadas y consistentes:** **ArgumentException** para datos inválidos, **InvalidOperationException** para reglas de negocio violadas; se capturan siempre en el mismo punto (cada opción del menú en **UI**), nunca a mitad de la lógica de negocio.
- **Sin duplicación de reglas de negocio:** la validación de fondos y la comisión existen en un solo lugar (**Member.RegisterWithdrawal**), no repetidas en **MovementService** ni en **TellerConsole**.
- **Separación estricta de responsabilidades:** ningún **Model** ni **Service** referencia **System.Console**; toda la E/S vive en **UI**.